

EmbedSim

EMBEDDED CONTROL · OPEN SOURCE

Differential Flatness Control

A Student Guide for IoT / AI Engineers

How to tune a motor controller when the real world gets noisy

Based on: `db42s02_closed_loop_dfc_20k.py` · EmbedSim open-source framework
github.com/vectorsim/embed_sim_project

Table of Contents

1	What the Controller Actually Does — in Plain English	3
2	The Three Knobs You Are Tuning	5
3	What Can Go Wrong — Noise, Load and Temperature	7
4	Why Manual Tuning Fails	10
5	The Tuner — How the Code Finds Better Gains	12
6	What the Cost Function Really Measures	16
7	Running the Tuner and Reading the Results	18
8	Your Thesis — What to Improve and Why It Matters	20
9	Deliverables and Thesis Claims	23
10	Your First Week — Day by Day	25
R	Reference: Motor Parameters and Key Papers	27

1 What the Controller Actually Does — in Plain English

1.1 The Motor as a Merry-Go-Round

Imagine a heavy playground merry-go-round. You want to spin it at exactly 2000 RPM and keep it there, even when children jump on and off. You can push it, but you must always push at exactly 90 degrees to the radius — any other angle and you waste effort.

A PMSM motor is that merry-go-round. The "push" is the magnetic force from the stator coils. The children jumping on are load torque steps. Your job as the controller is to keep the push angle exactly right, at exactly the right strength, 20,000 times per second.

1.2 Why Three-Phase Currents are Hard to Think About

The motor has three current wires. At 2000 RPM each current is a sine wave changing 133 times per second. Trying to control three sine waves simultaneously at 20 kHz would be like conducting three separate orchestras simultaneously in different time signatures — possible in principle, madness in practice.

The FOC transformation solves this with a change of perspective. Instead of standing in the lab watching three spinning sine waves, you *stand on the rotor* and look outward. From there, the rotating magnetic field appears stationary. The three AC currents become two DC values you can control directly with a simple proportional gain.

1.3 The Two DC Values — i_d and i_q

Signal	Name	Physical meaning	Control target
i_d	d-axis current	Current aligned with the rotor magnet — like pushing directly on the axle	$i_d = 0$ always. Non-zero i_d heats the motor without producing torque.
i_q	q-axis current	Current perpendicular to the rotor magnet — the actual push on the merry-go-round	$i_q = i_{q_ref}$, where i_{q_ref} is computed from the speed error.
ω	Shaft speed	How fast the rotor is spinning in rad/s (or RPM)	$\omega = \omega_ref = 2000$ RPM target

1.4 What "Differential Flatness" Actually Means

Differential flatness is a property of certain nonlinear systems. Before explaining what it means, it is worth understanding why nonlinearity matters for a PMSM motor in the first place.

A **linear** system is one where doubling the input exactly doubles the output. A simple resistor is linear: twice the voltage gives twice the current. A classical PI speed controller designed around a fixed operating point assumes the motor is linear near that point. This works adequately at one speed and one load, but the PMSM equations contain products of state variables — speed times current, current times current — that break the linearity assumption the moment conditions change. The same PI gains that give smooth control at 2000 RPM with no load can produce oscillation at 500 RPM under a heavy load.

A **nonlinear** controller deals with the true system equations rather than a linearised approximation. The price is complexity: nonlinear controllers typically require either a real-time solver (as in MPC)

or a very clever algebraic shortcut. Differential flatness is that shortcut. It works because the PMSM belongs to a special class of nonlinear system — one where you can choose certain output variables (the “flat outputs”) such that all other states and all inputs can be written explicitly in terms of those outputs and their derivatives. No simulation, no iteration required.

Think of a GPS route planner. It does not simulate every possible road one metre at a time. It computes the optimal route analytically and tells you exactly which turns to make. DFC is the GPS route planner for motor currents.

For this motor, the flat outputs are $[iq, \omega]$. Everything else — id , vd , vq — can be derived algebraically from iq , ω , and their time derivatives. This means:

- No iterative solver needed at 20 kHz (unlike MPC which solves a quadratic program every step).
- The feedforward voltage exactly cancels back-EMF, resistance drop, and cross-coupling — before any error has occurred.
- The feedback part only needs to handle what the model got wrong — noise, temperature drift, unknown load.
- The controller remains correct across the full speed and load range — it is not a linear approximation that degrades away from the design point. DFC at 500 RPM uses exactly the same structure as DFC at 3000 RPM; only the operating-point values inside the feedforward change.

Table 1.4 — Controller approaches compared

Aspect	Linear PI/PID	DFC (this guide)	MPC	SMC
System model	Linearised around one operating point	Full nonlinear; exact inverse used as feedforward	Linear model over short horizon	Nonlinear; sliding surface designed from full model
Computation at 20 kHz	~10 multiply-adds	~30 multiply-adds; closed form, no solver	QP solver; 100–1000 multiply-adds	~20 multiply-adds; sign function
Handles speed range variation	Degrades — gains must be retuned per operating point	Yes — feedforward adapts algebraically; three gains are global	Partially — horizon length may need adjustment	Yes — chattering may increase at low speed
Parameters to tune	Kp, Ki per loop — often 6 or more gains	3 gains only (Kp_speed, Kp_id, Kp_iq)	Q, R weight matrices — horizon N	Sliding gain, boundary layer — per axis

1.5 The Three-Line Controller

After all the mathematics, the actual control law in the code is remarkably simple:

```
# Speed loop → current demand
iq_ref = Kp_speed * (ω_ref - ω_meas)      # [A/(rad/s)] * [rad/s] = [A]

# d-axis: kill any flux you didn't ask for
vd = vd_feedforward + Kp_id * (0 - id)    # [V/A] * [A] = [V]

# q-axis: drive torque current to demand
vq = vq_feedforward + Kp_iq * (iq_ref - iq) # [V/A] * [A] = [V]
```

The feedforward terms (`vd_feedforward`, `vq_feedforward`) are computed analytically from the motor model and the current operating point — that is the flatness part. The three **Kp** gains are all that remains for you to tune.

1.6 Why This Matters for Your Career

You are not just tuning a motor controller. You are learning skills that companies pay for:

Control Theory + AI is a rare combination. Most engineers know one or the other. You will know both. The same pattern — surrogate modelling + gradient descent on a simulator — appears in:

- **Tesla's battery thermal management** — reduce cell degradation by 15% using surrogate-optimised charge curves
- **Boston Dynamics' gait optimisation** — walking patterns learned in simulation, then transferred to hardware
- **Google's data centre cooling** — 30% reduction in PUE via NN surrogate + gradient-free search on a thermal model
- **Every Formula E team's motor control** — that extra 0.5% efficiency wins races; gains are simulation-tuned before every event

The difference between a good engineer and a great one is knowing *when* to use AI and when to use physics. DFC is physics. The tuner is AI. You are the bridge.

2 The Three Knobs You Are Tuning

The entire tuning problem reduces to three proportional gains. Each gain is a single floating-point number. Together they determine how aggressively the controller reacts to errors in speed, d-axis current, and q-axis current.

The Three Parameters — from `_ACTIVE_GAINS` in the code

`Kp_speed` = 0.4 [A/(rad/s)] Speed error → torque current demand
`Kp_id` = 0.4 [V/A] d-axis current error → d-axis voltage correction
`Kp_iq` = 8.0 [V/A] q-axis current error → q-axis voltage correction

Search bounds used by the tuner:

`Kp_speed` : [0.05 , 0.60]
`Kp_id` : [0.20 , 6.00]
`Kp_iq` : [1.00 , 12.00]

2.1 `Kp_speed` — The Accelerator Pedal

`Kp_speed` converts a speed error into a current demand. Think of it as the sensitivity of the accelerator pedal: how much extra torque do you request for every 1 rad/s of speed shortfall?

At 2000 RPM the motor is spinning at 209 rad/s. If the actual speed is 190 rad/s, the speed error is 19 rad/s. With the default `Kp_speed` = 0.4, the controller demands $0.4 \times 19 = 7.6$ A of torque current. If that exceeds the motor's rated limit (3.35 A peak), the demand is clamped — the speed loop has saturated.

<code>Kp_speed</code>	Too small (< 0.1)	About right	Too large (> 0.6)
Speed tracking	Motor droops 50–200 RPM under load — feels like driving with the handbrake partly on	Tracks to within 2–5 RPM in steady state	Slams to saturation at every small error — current spikes cause acoustic buzzing
Load rejection	20+ RPM dip lasting > 200 ms when load steps from 5 to 20 mN·m	< 15 RPM dip, recovers in under 100 ms	Overshoots after load step, then oscillates before settling

2.2 `Kp_id` — The Flux Warden

`id` is the current component that produces no useful torque — only heat. The flatness feedforward should keep `id` close to zero automatically. `Kp_id` is the backup: if the feedforward is imperfect (because `R` has drifted with temperature, or an encoder glitch gave the wrong rotor angle), `Kp_id` corrects it.

Think of `Kp_id` as a zealous hall monitor. Set it low and small amounts of `id` linger unnoticed, quietly wasting energy. Set it too high and it panics at every ADC noise blip, injecting corrections that create oscillation.

K_p_{id} has a natural upper limit: the closed-loop bandwidth must stay below the switching frequency. K_p_{id} / L gives bandwidth in rad/s. With $L = 0.74 \text{ mH}$ and $K_p_{id} = 6.0$ (upper bound), bandwidth = $8100 \text{ rad/s} \approx 1.3 \text{ kHz}$ — still well below the 20 kHz switching frequency, but at the edge of prudent.

2.3 K_p_{iq} — The Torque Throttle

K_p_{iq} determines how fast the torque current tracks its reference. This is the gain that most directly affects responsiveness. A high K_p_{iq} means i_q tracks i_{q_ref} in microseconds; a low K_p_{iq} means it lags by milliseconds.

The default of 8.0 V/A gives a closed-loop bandwidth of $8.0 / 0.00074 \approx 10,800 \text{ rad/s} \approx 1.7 \text{ kHz}$ — fast relative to the mechanical time constant ($\approx 12 \text{ ms}$) but manageable with the 20 kHz ISR. In the presence of PWM switching spikes (which happen at 5% probability per step), a K_p_{iq} above ≈ 10 starts amplifying the spikes into the voltage commands.

2.4 How the Three Interact — the Triangle of Trouble

These three gains look independent in the control law, but they interact through the physics of the motor. Here is a concrete chain of consequences:

1. **K_p_{speed}** You increase to improve load rejection.
2. **i_{q_ref}** Larger speed errors now generate larger demands.
3. **$\omega_e \cdot L \cdot i_q$** Larger i_q creates a cross-coupling voltage on the d-axis.
4. The feedforward partially cancels this, but imperfectly at noisy speed estimates.
5. **i_d** Residual d-axis voltage pushes nonzero.
6. Now K_p_{id} needs to work harder to correct i_d , amplifying any ADC noise.
7. The extra v_d correction creates a small v_q disturbance through transformer coupling.
8. **K_p_{iq}** Your carefully tuned is now dealing with a different effective operating point.



The key insight

Changing one gain shifts the optimal value of the other two.

This is why manual trial-and-error tuning always converges to a mediocre local solution.

The AI tuner searches the full 3D space simultaneously — it finds trade-offs you cannot see by hand.

3 What Can Go Wrong — Noise, Load and Temperature

The simulation includes a realistic AURIX hardware noise model. Understanding each noise source — what causes it, how it enters the control loop, and what it does to your gains — is essential before you attempt tuning. These are not academic abstractions; every number here comes from the actual embedded hardware.

3.1 The Seven Gremlins in the Code

Gremlin	Magnitude	Where it enters	What it does to your gains
ADC quantisation noise	$\sigma = 2.6 \text{ mA}$ (1.5 LSB)	ia, ib, ic measurements	Corrupts Clarke transform → id and iq have high-freq ripple. Kp_id and Kp_iq amplify this into voltage noise.
ADC saturation clamp	Hard limit $\pm 4.5 \text{ A}$	ia, ib, ic measurements	Clips peak current during transients. High Kp_iq can demand voltages that push current into the clip region.
PWM switching spike	$\pm 0.5 \text{ A}$ impulse, 5% prob.	Phase-A current only	Creates a sudden false iq signal. Kp_iq > 10 amplifies spikes into vq pulses audible as ticking.
Encoder quantisation	1000 PPR × 4 = 1.57 mrad/count	Rotor angle θ	Corrupts Park/InvPark transforms. The feedforward uses wrong ω_e → residual id that Kp_id must fight.
Encoder glitch	± 2 counts, 0.2% prob.	Rotor angle θ	A sudden ± 3.14 mrad jump spikes the computed speed → large iq_ref spike → Kp_speed must not over-react.
Dead-time distortion	136 mV per phase (400 ns)	$\alpha\beta$ voltage commands	Systematic voltage error proportional to current sign. Creates a small id offset that looks like wrong Kp_id.
DC bus ripple	$\pm 0.5 \text{ V}$ @ 100 Hz	SVPWM modulation index	Modulation index fluctuates. At high speed the motor is close to the voltage limit; ripple can push it over.

3.2 Load Torque — the Children on the Merry-Go-Round

The load schedule in the code applies two step changes to the motor shaft:

```

t < 0.5 s : T_load = 0.000 N·m (free-spinning, no load)
0.5 - 1.2 s : T_load = 0.005 N·m (light load ~ pencil eraser pressing on shaft)
t > 1.2 s : T_load = 0.020 N·m (heavy load ~ 4× light load)

```

The heavy load step at $t = 1.2 \text{ s}$ is the controller's hardest test. It demands $0.020 / 0.0168 \approx 1.19 \text{ A}$ of extra iq immediately. The speed loop must detect the speed drop, increase iq_ref, and wait for the q-axis current loop to respond — all within the inertia budget before the speed error becomes uncomfortable.

▲ The Load Rejection Chain

$t = 1.200 \text{ s}$: load applied → motor begins to decelerate

t = 1.200 s : within 50 μ s (one ISR tick), speed error grows $\rightarrow$ iq_ref increases
t = 1.201 s : Kp_iq drives extra vq $\rightarrow$ iq rises toward new iq_ref
t = 1.203 s : extra torque current delivered $\rightarrow$ deceleration arrested
t = 1.210 s : speed returns toward 2000 RPM

The speed dip between 1.200 and 1.210 s is the metric load_drop.

Kp_speed and Kp_iq both influence it — but in opposite directions for noise.

3.3 Temperature Drift — the Slow Invisible Enemy

The motor's phase resistance R starts at 2.35 Ω . At 60°C above ambient it rises to approximately 2.91 Ω — a 24% increase. The DFC feedforward uses the nominal R value. When the motor is hot, the feedforward under-compensates the resistive voltage drop, creating a persistent id error that Kp_id must correct.

This is why id_rms (the RMS value of id in steady state) is in the cost function with such a heavy weight: a hot motor with poorly tuned Kp_id will waste energy continuously.

Your thesis should test tuned gains at R +24% to verify the gains remain robust. A gain set that achieves cost = 18 at nominal temperature but cost = 45 at operating temperature is not production-ready.

4 Why Manual Tuning Fails

4.1 The Fog of Three Dimensions

Imagine you are blindfolded, standing somewhere in a hilly landscape. You can take one step in any direction, then measure whether you went up or down. Your goal is to find the lowest valley.

With one parameter (one dimension), this is easy — walk downhill and stop. With three parameters, the valley might be "northeast" in the Kp_speed/Kp_iq plane but simultaneously northwest in the Kp_id/Kp_iq plane. You cannot see this from any single slice.

Manual tuning works one gain at a time: fix two, adjust one, repeat. This is called coordinate descent, and on a curved, correlated landscape it reliably finds local minima rather than global ones. You might spend hours getting to cost = 45 when cost = 18 is achievable fifteen minutes away in the parameter space.

Coordinate descent works well for **linear** systems where each parameter affects the output independently. A classical PI current loop in isolation is nearly linear — Kp_iq changes the bandwidth, Ki_iq shifts the steady-state error, and the two barely interact. You can tune them one at a time and get a near-optimal result. The DFC cost landscape is different: it is generated by a **nonlinear** plant (the PMSM motor equations) evaluated through a stochastic noise model. The cost iso-surfaces in the 3D (Kp_speed , Kp_id , Kp_iq) space are tilted, curved, and may have narrow ridges that coordinate descent slides off without ever detecting. The AI tuner treats the entire 3D space as a single object to minimise — which is the only correct approach for a coupled nonlinear system.

4.2 The 5-Second Simulation Tax

Each simulation runs for 5 simulated seconds at 20 kHz — 100,000 integration steps. On a modern laptop that takes roughly 40–60 seconds of wall-clock time. If you manually try 20 gain combinations, that is 20 minutes of waiting between experiments. Boredom and impatience drive you toward premature convergence: you stop exploring when you find something "good enough" rather than good.

The tuner automates this process and runs 40 structured explorations (Phase 1) plus a verification run — about 41 simulations total. That is roughly 30–40 minutes of machine time while you do something else, producing a gain set that has been rigorously compared against 40 alternatives across the full noise model.

4.3 The Interaction Trap

Here is a concrete example of why one-at-a-time tuning fails. Suppose you have found a good $Kp_iq = 8.0$ and are now tuning Kp_speed . You test $Kp_speed = 0.6$ and find it causes speed oscillation. You conclude: " Kp_speed should not exceed 0.5."

But here is what you missed: with $Kp_iq = 10.0$ instead of 8.0, the faster torque current response damps out the oscillation, making $Kp_speed = 0.6$ perfectly stable — and producing lower cost than your "safe" $Kp_speed = 0.5$ with $Kp_iq = 8.0$. You never tested that region because you tuned the gains sequentially, not simultaneously.

Metaphor: tuning a piano versus tuning a guitar

A guitar can be tuned string by string — each string's tension barely affects the others.

A piano cannot. Changing the tension of one string shifts the soundboard, which changes the effective tension of adjacent strings.

The three DFC gains are a piano, not a guitar.

You must tune all three simultaneously — or accept a mediocre result.

4.4 Noise Changes the Answer

With no noise, the optimal gains are approximately $Kp_speed = 0.55$, $Kp_id = 2.0$, $Kp_iq = 10.0$. In the presence of the AURIX noise model, these same gains produce excessive iq chattering — the noise floor gets amplified through the high Kp_iq into the voltage commands.

The optimal gains under the real noise model are different from the optimal gains in the idealised model. If you tune manually in a clean simulation and then deploy to hardware, the gains are wrong before you even start.

4.5 The Psychological Trap — Why Your Brain Lies to You

Manual tuning feels productive. You change a gain, run a simulation, see a result. Each iteration takes 40 seconds. After 10 iterations you have “invested” 7 minutes. Your brain interprets this as progress.

But here is what actually happens in those 10 iterations:

#	What you changed	What you learned
1	$Kp_speed\ 0.4 \rightarrow 0.5$	Speed tracking improved
2	$Kp_speed\ 0.5 \rightarrow 0.55$	Slight oscillation at load step
3	Kp_speed back to 0.48	<i>“Good enough”</i>
4	$Kp_iq\ 8.0 \rightarrow 9.0$	No visible change
5	$Kp_iq\ 9.0 \rightarrow 10.0$	Slight buzz
6	Kp_iq back to 8.5	<i>“Seems fine”</i>
7	$Kp_id\ 0.4 \rightarrow 1.0$	Nothing obvious
8	$Kp_id\ 1.0 \rightarrow 2.0$	Still nothing
9	Kp_id back to 0.4	<i>“It was fine before”</i>
10	$Kp_speed\ 0.48 \rightarrow 0.52$	Oscillation returns

The truth you missed: the optimal point is $Kp_speed = 0.51$, $Kp_id = 2.3$, $Kp_iq = 9.8$. You never got close. But you *feel* like you tried everything.

The local search trap

Humans are excellent at hill climbing — taking the next step that feels downhill. But in a 3D nonlinear cost landscape, “feels downhill” is almost never the same as “leads to the global minimum.” The AI tuner does not feel anything. It systematically covers the space, finds all the low valleys, and returns the best one. You explore randomly. The AI wins every time.

The 40-second simulation tax is a feature, not a bug. It makes manual exploration expensive enough that you are forced to use AI.

5 The Tuner — How the Code Finds Better Gains

The tuner (`run_tuner()` in the simulation file) runs in four phases. Each phase builds on the previous one. Understanding each phase is required before you can improve it for your thesis.

5.1 Phase 1 — Latin Hypercube Sampling: Casting a Wide Net

The first problem is: how do you choose 40 gain combinations to evaluate without clustering them all in one corner of the 3D space? Random sampling is wasteful — you might pick ten points near $Kp_speed = 0.3$ and none near 0.5.

Latin Hypercube Sampling (LHS) solves this like a very organised bus timetable. It divides each axis into 40 equal bins and guarantees exactly one sample per bin per axis. The result: the 40 simulations are spread evenly across the entire search space, with no large unexplored regions.

```
# From the code: _t_lhs(n, bounds, rng)
for each dimension (Kp_speed, Kp_id, Kp_iq):
    divide axis into n equal bins
    draw one sample uniformly within each bin
    shuffle the order across dimensions ← breaks correlations
```

The shuffle step is important. Without it, sample 1 would have the lowest Kp_speed AND the lowest Kp_id AND the lowest Kp_iq . With the shuffle, each sample is a genuinely diverse combination. This is what makes 40 samples informative in a 3D space.

5.1a Python Libraries for LHS — What the Code Actually Uses

The EmbedSim tuner implements LHS from scratch in `_t_lhs()` using only NumPy — no external sampling library required. This keeps the dependency footprint minimal for embedded-world engineers who may not have access to optional SciPy submodules. However, for your thesis work you have two well-supported alternatives that offer additional features:

- **scipy.stats.qmc.LatinHypercube** (SciPy ≥ 1.7) — the standard choice for new thesis work. Produces an optimal LHS with a `centered=True` option (each sample at the bin centre rather than a random position within the bin) and an optional `optimization` flag that maximises minimum inter-sample distance (“maximin LHS”) for even better coverage.
- **pyDOE2** — a standalone library (`pip install pyDOE2`) that also supports full factorial, Box-Behnken, and central composite designs. Useful if your thesis extends the tuner to design-of-experiment (DoE) methodology.

The following snippet shows how to replace the built-in `_t_lhs()` with the SciPy version (one-line swap, same output shape):

```
# Option A — SciPy (recommended for thesis)
from scipy.stats.qmc import LatinHypercube, scale

# bounds = [(lo, hi), (lo, hi), (lo, hi)] — same format as _t_lhs()
sampler = LatinHypercube(d=len(bounds), seed=42, optimization="random-cd")
unit_samples = sampler.random(n=n) # shape (n, d), values in [0, 1]
l_bounds = [b[0] for b in bounds]
u_bounds = [b[1] for b in bounds]
samples = scale(unit_samples, l_bounds, u_bounds) # shape (n, d), in physical
units
```

The `optimization="random-cd"` flag runs a centred-discrepancy optimisation that spreads the samples more uniformly than a plain shuffle. For 40 samples in 3D the difference is modest; for 20 samples in 5D (e.g., if you add Kp_id_i and Kp_iq_i integral gains to your search) it becomes

significant. The `seed=42` argument makes the exploration reproducible — essential for a thesis where another researcher must be able to re-run Phase 1 and obtain the same 40 sample points.

Metaphor: sampling restaurants in a city

Random sampling: you might pick 5 Italian restaurants by chance and miss all the Thai.

Latin Hypercube: you guarantee one restaurant from each of 40 city grid squares.

Every area of the city is represented. No surprises when you deploy.

5.2 Phase 2 — Training the Neural Network Surrogate

After 40 simulations you have 40 (gains, cost) pairs. Each simulation took ~40 seconds. Now the question is: can you predict what cost a new set of gains would produce, without running another 40-second simulation?

The neural network is trained to do exactly this. It learns the shape of the 3D cost landscape from the 40 data points — a compact, smooth, differentiable approximation.

```
# Architecture: 3 inputs → 16 hidden → 16 hidden → 1 output
#               (tanh activations, linear output)
#
# Input:  normalised gain vector [Kp_speed, Kp_id, Kp_iq] → [0,1]^3
# Output: normalised cost (standardised to mean=0, std=1)
#
# Training: Adam optimiser, 500 epochs, learning rate = 0.003
```

Why tanh and not ReLU? ReLU activations create piecewise-linear functions — they approximate the landscape with a mosaic of flat triangular facets. For a **linear** plant — where cost varies smoothly and predictably with gains — this is often acceptable. For a **nonlinear** plant with a stochastic noise model, the cost landscape has gentle curvature and no sharp corners — exactly the shape that tanh captures well. Tanh activations produce a network whose gradient (used in Phase 3) is smooth and well-conditioned everywhere in the search space. A ReLU surrogate would produce a Phase 3 gradient that is constant within each facet and undefined at facet boundaries, causing the gradient descent to take erratic steps.

Why 16 hidden units? With 40 training points and 3 inputs, you need a network that generalises rather than memorises. The code includes a comment explaining the calculation: 16 units gives 353 parameters, which is still underdetermined relative to 40 samples — the network is forced to find the smooth trend rather than fitting every point exactly.

Network choice	Parameters	Behaviour with 40 pts	Verdict
3 → 8 → 8 → 1	≈ 100	May underfit — cannot capture curvature	Too simple for a curved 3D landscape
3 → 16 → 16 → 1 (current)	353	Good balance — generalises the trend	Correct choice for 40 samples
3 → 64 → 64 → 1	≈ 4,700	Memorises noise in training data	Overfits with only 40 samples

After training, the code measures R^2 on a held-out validation set (20% of the 40 points). An $R^2 > 0.70$ means the network has learned the landscape meaningfully. If $R^2 < 0.50$, the surrogate is unreliable and more Phase 1 samples are needed.

5.3 Phase 3 — Gradient Descent on the Network's Input

Here is the clever trick. The neural network is differentiable. You can compute the gradient of its output with respect to its input — analytically, in microseconds. This means you can run gradient descent not to train the network, but to find the input that minimises the network's output.

Think of it as this: you spent 40×40 seconds = 27 minutes building a model of the terrain. Now you put a marble on the model and let it roll downhill. The marble finds the low point in seconds.

```
# Phase 3: gradient descent on the network INPUT (not weights)
for restart in range(8): # 8 independent starting points
    x = random_start_in_[0,1]^3
    for step in range(400):
        cost, gradient = mlp.scalar_grad(x) # exact backprop gradient
        x = x - lr * Adam_update(gradient) # gradient descent step
        x = clip(x, 0, 1) # stay inside valid bounds
        record(x, cost)
    return x with lowest predicted cost
```

Eight independent restarts guard against local minima — the marble is dropped from eight different starting positions, and the lowest final position wins. Restart 0 always starts from the best of the 40 Phase 1 samples (warm start); restarts 1–7 start randomly.

5.4 Phase 4 — Verification: Does the Real World Agree?

The neural network's prediction is not guaranteed to be correct. The network learned from 40 noisy data points. Its predicted optimum might lie in a region that was not sampled well, or the actual cost landscape might have non-smoothness the network did not capture.

Phase 4 runs one real simulation at the predicted optimum, with the full AURIX noise model. If the verified cost is lower than the best Phase 1 result, the NN-predicted gains win. If not, the best Phase 1 gains are used instead — the fallback is always safe.

Safety net — from the code

```
if met_verify['cost'] < best_obs_cost:
    final_gains = gains_pred ← neural network found something better
else:
    final_gains = best_obs_gains ← observed best is still champion
```

The tuner can never produce worse gains than the best of the 40 LHS samples.

The NN is an accelerator, not a gamble.

5.5 Phase 5 — Writing the Gains Header

If the verified result is acceptable, the tuner writes an MISRA C:2012-compliant header file (`embed_sim_dfc_gains_tuned.h`) containing the optimal gains with a full audit trail: tuning date, noise model parameters, cost achieved, and the physics-derived bandwidth for each gain. This header can be compiled directly into the AURIX firmware — no manual transcription needed.

6 What the Cost Function Really Measures

The cost function is the tuner's only compass. Every trade-off in the final gains was decided by the weights in this function. Before you change anything in your thesis, you must understand what each term is actually measuring and why it is weighted the way it is.

6.1 The Formula

```
J = 2.0 * ss_err [RPM]      ← steady-state speed accuracy
  + 50.0 * id_rms [A]       ← d-axis current leakage (energy waste)
  + 4.0 * iq_chat [A]       ← q-axis current chattering (noise rejection)
  + 20.0 [penalty]         ← if modulation index p90 >= 0.93
```

Evaluated over the final 15% of the 5-second simulation
(i.e. from $t = 4.25$ s to $t = 5.0$ s — well past transients and load steps)

6.2 Why `id_rms` is Weighted 25× More Than Speed Error

This surprises most students. Speed accuracy seems like the main objective — why is `id_rms` penalised so much more heavily?

The answer is frequency. A 1 RPM speed error means the motor is travelling at 1999 instead of 2000 RPM — negligible in most applications. But `id = 0.05 A` means 0.05 A of wasted current flowing through $2.35\ \Omega$ of resistance, creating 5.9 mW of heat — at every one of the 20,000 ISR ticks per second, every second the motor runs. Over an 8-hour production shift, this is 170 Wh of unnecessary heat, potentially requiring more expensive thermal management or reducing motor lifetime.

The weight of 50 reflects this economic reality: a small improvement in `id_rms` is worth far more than a small improvement in steady-state speed accuracy.

6.3 The Over-Modulation Penalty

The SVPWM modulator has a physical ceiling. When the commanded voltage magnitude exceeds the inscribed circle of the hexagonal voltage space, the output clips — the voltage you asked for is not the voltage you get. At that point the control law breaks down.

The penalty of +20 is applied whenever the 90th-percentile modulation index exceeds 0.93. This is a hard penalty, not a proportional one: a gain set that occasionally hits the voltage ceiling is worse than one that stays safely below it, regardless of how well it performs on the other metrics.

6.4 Performance Levels

Level	ss_err [RPM]	id_rms [A]	iq_chat [A]	Total J
Excellent	< 2	< 0.01	< 0.05	≈ 4–8
Good	< 5	< 0.03	< 0.15	≈ 15–25
Acceptable	< 10	< 0.05	< 0.30	≈ 30–45
Poor — revise	> 20	> 0.10	> 0.50	> 60

gains				
Thesis target	< 5	< 0.03	< 0.15	< 22

6.5 What the Cost Function Cannot See

This is a critical section for your thesis. Every objective that is not in the cost function is invisible to the tuner. The current cost function does not penalise:

- Load drop magnitude (RPM dip at the heavy load step at $t = 1.2$ s)
- Transient overshoot during the initial ramp from 0 to 2000 RPM
- Current during acceleration (the tuner only evaluates the final 15% — steady state)
- Motor temperature (resistance drift — the tuner uses nominal parameters)

If your deployment cares about any of these — and for an IoT application (drone, pump, conveyor) the load drop usually matters a great deal — your thesis should add the appropriate term to the cost function. Section 8 describes how.

7 Running the Tuner and Reading the Results

7.1 Command Line

```
# Skip tuner – run with default hardware-commissioning gains
$ python db42s02_closed_loop_dfc_20k.py --no-tune

# Run tuner, then run main simulation with tuned gains
$ python db42s02_closed_loop_dfc_20k.py --tune

# Interactive – prompts 'Run tuner? [y/N]'
$ python db42s02_closed_loop_dfc_20k.py
```

7.2 What You Will See During a Tuner Run

```
[Phase 1] LHS exploration ...
[ 1/40] Kp_speed=0.1200 Kp_id=0.890 Kp_iq=3.12 -> cost=42.1 ss=8 RPM
id=0.041 A
[ 2/40] Kp_speed=0.5300 Kp_id=4.320 Kp_iq=9.87 -> cost=18.4 ss=3 RPM
id=0.011 A
...
Best observed: cost=14.2

[Phase 2] Training MLP surrogate ...
epoch    0  MSE=0.842311
epoch   100  MSE=0.023415
epoch   500  MSE=0.004127
Surrogate R^2 (held-out 8 pts): 0.8341 (>0.70 is acceptable)

[Phase 3] Gradient descent on surrogate (8 restarts) ...
Surrogate minimum: predicted cost = 11.7
Kp_speed  = 0.4821
Kp_id     = 2.3105
Kp_iq     = 9.2340

[Phase 4] Verification simulation ...
NN gains BETTER: cost 13.1 < 14.2
```

7.3 Reading the Nine Output Plots

Plot	What to look for	Red flag → likely cause
Speed [RPM]	Smooth ramp, <5 RPM overshoot, small dip at load step, flat in SS	> 20 RPM dip at t=1.2 s → Kp_speed too small or Kp_iq too small
q-axis current [A]	iq tracks iq_ref within 2 ms, small ripple in steady state	iq lags iq_ref by > 5 ms → Kp_iq too small
d-axis current [A]	Flat near 0 A throughout. Tiny bump at load step (< 0.1 A, < 10 ms)	> 0.05 A steady-state → Kp_id too small; high ripple → Kp_id too large
$\alpha\beta$ voltages [V]	Smooth sinusoids, well within ± 17 V at all times	Clipping → voltage saturation → gains need reducing or trajectory smoother ramp

SVPWM Vref [0–1]	Should stay below 0.93 in steady state (the over-mod threshold)	Persistently at 1.0 → motor too close to voltage limit at this RPM
SpeedFusion α	Rises from 0 (encoder) to ~1 (SMO) as speed reaches rated. Smooth transition.	Sudden step changes → encoder glitch interacting with fusion threshold
ia ADC noise	Noisy band ± 0.5 A around the ideal current — normal	Spikes $> \pm 1$ A frequently → PWM spike rate unusually high (check <code>_SPIKE_PROB</code>)
Encoder error [mrad]	Within ± 1.57 mrad (one count) except rare ± 3.14 mrad glitches	Systematic offset → encoder zero calibration issue in real hardware
DC bus ripple [V]	± 0.5 V sinusoid at 100 Hz around 17 V nominal	Amplitude > 1 V → capacitor bank sizing issue in real hardware

8 Your Thesis — What to Improve and Why It Matters

The baseline tuner works. Your thesis task is to make it work better — faster, more robustly, or across a wider range of conditions. Below are five concrete improvements, ordered from most accessible to most ambitious. Each is grounded in the actual code.

8.1 Improvement A — Add Load Drop to the Cost Function

This is the highest-impact change you can make with the least code. The current cost function is blind to transient performance — it only looks at the final 15% of the simulation, missing everything that happens at the load steps.

The load drop (RPM dip at $t = 1.2$ s) is the metric that most users of an IoT motor care about. A pump that drops 80 RPM when a valve opens will cavitate. A drone that drops 80 RPM on a gust will pitch.

```
# Current cost function (evaluates only steady state)
ss = t > 0.85 * T_SIM
cost = 2.0*ss_err + 50.0*id_rms + 4.0*iq_chat

# Proposed addition: measure load drop at t = 1.2 s
step_mask = (t >= 1.18) & (t <= 1.35)    # window around heavy load step
speed_at_step = rpm[step_mask]
load_drop_rpm = TARGET_RPM - np.min(speed_at_step)

cost += 1.5 * load_drop_rpm                # suggested weight
```

Your thesis should: (1) add this term, (2) re-run the tuner, (3) compare the new gains with the old ones. You will likely find that K_p speed increases and K_p iq increases — the tuner now cares about transient response, not only steady state.

8.2 Improvement B — Replace LHS with Bayesian Optimisation

The 40-simulation Phase 1 is the most time-consuming part of the tuner. It is effective but dumb: it does not use the cost values it is discovering to decide where to sample next. A smarter approach uses Bayesian Optimisation.

The idea: after each simulation, update a Gaussian Process model of the cost landscape. Use the model to choose the next sample point that has the highest expected improvement over the current best — the "Expected Improvement" acquisition function. This concentrates simulations in promising regions rather than spreading them uniformly.

Why Bayesian Optimisation outperforms LHS here

LHS after 10 samples: knows 10 points, next sample ignores them.

BO after 10 samples: has a probabilistic map of the entire landscape,
chooses the 11th sample where improvement is most expected.

Typical result: BO reaches the same cost as LHS in 20 samples instead of 40.

That is a 50% reduction in tuning time — from ~30 minutes to ~15 minutes.

The `scikit-learn` library provides a `GaussianProcessRegressor`. The `scipy.stats.norm` provides the normal CDF needed for Expected Improvement. The implementation is approximately 50 lines of Python — achievable in one day.

8.3 Improvement C — Multi-Point Robustness Evaluation

The current tuner evaluates gains at one operating point: 2000 RPM, fixed load schedule. But your IoT deployment will run at many speeds and loads. A gain set that achieves cost = 14 at 2000 RPM might achieve cost = 60 at 500 RPM (where back-EMF is low and the feedforward changes character), making it useless for the full operating envelope.

```
# Current: one operating point
cost = _t_run_with_gains(Kp_speed, Kp_id, Kp_iq, target_rpm=2000)

# Proposed: worst-case across three operating points
costs = []
for rpm_target in [500, 1000, 2000]:
    c = _t_run_with_gains(Kp_speed, Kp_id, Kp_iq, target_rpm=rpm_target)
    costs.append(c)
cost = max(costs) # minimax: optimise the worst case
```

This triples the simulation time per evaluation ($3 \times 40 \text{ s} \approx 2 \text{ min}$ per LHS sample). With 40 samples, that is 80 minutes of Phase 1. Combine this with Improvement B (BO) to reduce to 20 samples — and you are back to 40 minutes while achieving robustness across the operating range.

8.4 Improvement D — Quantify Surrogate Uncertainty

The current MLP gives one cost prediction for each gain vector. It cannot say "I am confident about this prediction" versus "I have not seen many samples near here." Adding uncertainty quantification would make Phase 3 more intelligent: instead of gradient-descending on the predicted cost, descend on the *lower confidence bound* — favouring regions the network is both optimistic about and uncertain about.

The simplest way to add uncertainty: train an ensemble of 5 MLPs (each with a different random seed). The variance of their predictions is a proxy for uncertainty. This is 5 lines of extra code in Phase 2 and a minor change to Phase 3.

```
# Phase 2: train ensemble
mlps = [_MLP(seed=i) for i in range(5)]
for mlp in mlps:
    mlp.train(X_tr, y_tr, epochs=500)

# Phase 3: gradient descent on lower confidence bound
def lcb(x, kappa=1.5):
    preds = [mlp.forward(x[None,:])[0] for mlp in mlps]
    return np.mean(preds) - kappa * np.std(preds)
```

8.5 Improvement E — Adaptive Noise Level Tuning (Advanced)

Currently the tuner always runs Phase 1 with the full AURIX noise model. This means every simulation has some variability — the stochastic noise can shift the cost by ± 3 units between identical gain sets. This noise in the cost function confuses the surrogate.

A progressive noise strategy runs Phase 1 with reduced noise (faster, more stable cost estimates) and Phase 4 verification with full noise. The key question for your thesis: at what noise level does the Phase 1 landscape best predict the Phase 4 (full noise) landscape? This is a mini research question that can be answered in 3 plots.

9 Deliverables and Thesis Claims

File	Contents	Why it matters
db42s02_closed_loop_dfc_20k.py (modified)	Your cost function improvements, BO or ensemble changes	Reproducible — any marker can re-run the tuner
embed_sim_dfc_gains_tune.d.h	MISRA C:2012 header with tuned gains + full audit trail	Drop-in firmware replacement — ISO 26262 traceable
results/phase1_scatter.png	3D scatter: 40 LHS points coloured by cost	Visualises the cost landscape — required for Chapter 4
results/convergence.png	Best cost vs evaluation number for your method vs baseline LHS	Quantifies improvement — the core thesis plot
results/test_matrix.csv	Tuned gains evaluated at 500, 1000, 2000 RPM and $\pm 24\%$ R drift	Proves robustness — not just one operating point
thesis/chapter_4.pdf	Method, results, comparison table, convergence plots	Academic contribution

CLAIM 1 — Efficiency

"The [your method] tuner reaches cost < 20 in $[N]$ simulations, compared to $[N_baseline]$ for the baseline LHS approach — a $[X]\%$ reduction in tuning time."

Evidence: convergence curve (cost vs evaluation number), 5 independent runs.

CLAIM 2 — Load Robustness (if you implement Improvement A)

"Gains tuned with the augmented cost function (including load_drop term) reduce the heavy-load RPM dip from $[X]$ RPM to $[Y]$ RPM at no cost to steady-state accuracy."

Evidence: speed plot comparison, before/after cost function.

CLAIM 3 — Operating Range Robustness (if you implement Improvement C)

"Multi-point tuned gains achieve cost < 22 at 500 RPM, 1000 RPM, and 2000 RPM. Baseline single-point tuned gains achieve cost < 22 only at 2000 RPM."

Evidence: test_matrix.csv showing cost at each operating point.

10 Your First Week — Step by Step

Do not be intimidated. Every day ends with something you can show. By Friday you will have reproduced the full tuner pipeline and have a baseline result to improve upon.

Step 1 — Understand the Baseline

9. **Run** `$ python db42s02_closed_loop_dfc_20k.py --no-tune`. The simulation completes in about 40 seconds. Observe the nine plots. Write down: what is the speed error, `id_rms`, and `iq` chattering with the default gains?
10. **_ACTIVE_GAINS** Open the code and find `_ACTIVE_GAINS`. Manually change `Kp_speed` to 0.1. Re-run. What changes? The speed droops — you have just seen the effect of one gain in isolation.
11. Change `Kp_iq` to 1.0. Re-run. Now `iq` lags `iq_ref` visibly. Restore to 8.0 before Day 2.

Step 2 — Understand the Cost Function

12. **_t_cost()** Open `_t_cost()` and read through it. Find where `ss_err`, `id_rms`, and `iq_chat` are computed. Add a print statement that shows each component separately, not just the total.
13. Run with the default gains. Write down the three cost components and their weighted contributions. Which term dominates?
14. Try five gain sets manually (change one gain at a time). Plot cost vs that gain. You should see a U-shaped or monotone curve for each. This is the landscape slice your AI must navigate in 3D.

Step 3 — Reproduce Phase 1

15. **_T_N_EXPLORE = 10** Set `_T_N_EXPLORE = 10` (reduce from 40 to save time). Run with `--tune`. Watch the Phase 1 output print for each simulation.
16. After Phase 1, add a plot: scatter the 10 gain sets in (`Kp_speed`, `Kp_iq`) space, coloured by cost. Can you see a trend? The low-cost region should be visible even with only 10 points.
17. **_T_N_EXPLORE = 40** Restore `_T_N_EXPLORE = 40` before continuing.

Step 4 — Understand the Neural Network

18. After a full 40-sample run, save the (`X_norm`, `y_norm`) arrays to a CSV. Load them in a notebook. Plot predicted cost vs actual cost for the held-out validation set. The R^2 value printed by the tuner corresponds to what you see.
19. **_MLP.__init__** Intentionally reduce hidden size to 4 (edit `_MLP.__init__`). Retrain. Does R^2 fall? Increase to 64. Does it improve on training set but not on validation? You have just demonstrated overfitting and underfitting experimentally.

Step 5 — Full Pipeline and First Improvement

20. Run the complete tuner with the full 40-sample Phase 1. Record the final gains and cost.
21. Add the `load_drop` term to the cost function (Improvement A, Section 8.1). Re-run the tuner. Compare the new gains with the old ones.

22. Run both gain sets against the default simulation and compare the speed dip at $t = 1.2$ s in the plots. You have now completed a meaningful research comparison.

You are not adjusting knobs until the numbers look acceptable.

You are teaching a machine to navigate a landscape you can barely visualize.

That is what makes this worth doing.

R Reference: Motor Parameters and Key Papers

DB42S02 Motor Parameters (from `smc_controller_block.DB42S02`)

Parameter	Symbol	Value	Notes
Phase resistance	<code>R_S</code>	2.35 Ω	+24% at 60°C above nominal — <code>Kp_id</code> robustness target
d-axis inductance	<code>L_D</code>	0.74 mH	$\tau_e = L/R = 315 \mu s$. <code>Kp_iq</code> BW = <code>Kp_iq</code> / <code>L_D</code>
Flux linkage	λ_{PM}	0.0014 Wb	Back-EMF = $p \cdot \lambda \cdot \omega_{mech}$. At 2000 RPM: 0.44 V
Pole pairs	<code>P_POLES</code>	4	$\omega_e = P_POLES \cdot \omega_{mech}$
Rated current (peak)	<code>I_MAX</code>	3.57 A	Hard saturation limit for <code>iq_ref</code>
DC bus voltage	<code>V_DC</code>	17 V	+/-0.5 V ripple at 100 Hz modelled in simulation
ISR period	<code>DT</code>	50 μs	20 kHz switching frequency = 100,000 steps per 5 s simulation

Key References

Reference	Why it matters for your thesis
Fliess, M. et al. (1995) — Sur les systèmes non linéaires différentiellement plats, C.R. Acad. Sci.	Original flatness paper — defines the theory your controller implements
Levine, J. (2009) — Analysis and Control of Nonlinear Systems, Springer	Best textbook for flatness control — Chapter 3 covers motor applications
Snoek, J. et al. (2012) — Practical Bayesian Optimisation (NeurIPS)	Foundation of Improvement B — Gaussian Process surrogate + EI acquisition
Hansen, N. (2016) — The CMA-ES Tutorial (arXiv:1604.00772)	CMA-ES: an alternative to LHS + NN for Phase 1 — useful comparison baseline
Kingma, D. & Ba, J. (2015) — Adam: A Method for Stochastic Optimisation (arXiv:1412.6980)	The optimiser used for both MLP training and surrogate gradient descent
EmbedSim — github.com/vectorsim/embed_sim_project	The simulation framework — contains all motor parameters and noise constants used in this guide